

SAGE (Semantic Analysis and Generation Engine): A Cross-Source Reasoning Engine over Heterogeneous Knowledgebases

Srujan Zanjali

Computer Science and Engineering
St. Vincent Pallotti College
of Engineering and Technology
Nagpur, Maharashtra, India
zanjalsrujan94@gmail.com

Rimanshu Sonule

Computer Science and Engineering
St. Vincent Pallotti College
of Engineering and Technology
Nagpur, Maharashtra, India
rimanshusonule00@gmail.com

Pratik Bhoyar

Computer Science and Engineering
St. Vincent Pallotti College
of Engineering and Technology
Nagpur, Maharashtra, India
pratikbhoyar156@gmail.com

Tanisha Shingne

Computer Science and Engineering
St. Vincent Pallotti College
of Engineering and Technology
Nagpur, Maharashtra, India
tanishashingne04@gmail.com

Prof. Ayaz Khan

Computer Science and Engineering
St. Vincent Pallotti College
of Engineering and Technology
Nagpur, Maharashtra, India
ayazahmedkhan.ngp@gmail.com

Abstract—Most retrieval-augmented generation (RAG) systems are built and evaluated over a single, homogeneous corpus of text. Practical knowledge work seldom fits that assumption. A user investigating a topic may accumulate a project website, several PDFs, a recorded lecture, and a code repository, then expect to query all of them through one conversational interface. We present SAGE (Semantic Analysis and Generation Engine), a system that ingests four structurally different source types (crawled websites, PDF documents, YouTube video transcripts, and GitHub code repositories) through a shared ingest-chunk-embed-index contract, and answers questions over them with a single retrieval-reranking-generation pipeline. Three design decisions anchor the system. First, a deterministic, non-generative source-overview chunk makes broad questions retrievable without a second LLM call. Second, a hybrid semantic-lexical reranking formula, paired with a diversity floor, guarantees fair citation representation across multiple sources. Third, a confidence heuristic gates refusal in a citation-only Grounded Mode, as opposed to an always-answering exploratory mode. We validate the system through its test suite and an isolated, end-to-end evaluation harness that exercises the unmodified production code against three live-ingested sources over fourteen query executions. A concrete design defect, the silent starvation of one source’s evidence in multi-source queries, surfaced during testing and is corrected by the diversity floor. SAGE contributes a checkable set of engineering decisions for grounded QA over heterogeneous, real-world knowledgebases; it introduces no new retrieval or generation model, and we document its limitations candidly.

Index Terms—retrieval-augmented generation, multi-source retrieval, question answering, reranking, chunking, confidence estimation

I. Introduction

This paper addresses grounded question answering over a knowledgebase whose contents are structurally heterogeneous: web pages, documents, video transcripts, and

source code coexist and must be queried together. A retrieval-augmented generation (RAG) pipeline conditions the language model’s answer on passages retrieved from an external corpus [1]. Published RAG systems are, with few exceptions, designed and measured against a single textual corpus of uniform structure. Real knowledgebases are rarely that clean: a query must be routed to whichever source actually holds the relevant evidence and return a citation naming where it originated, forcing extraction, chunking, retrieval, and citation to behave as one contract across source types that share almost nothing in their native structure. That contract, rather than any individual retrieval or generation model, is the engineering problem this paper takes on.

SAGE (Semantic Analysis and Generation Engine) is such a system. It ingests four source types, crawled multi-page websites, PDF documents, YouTube video transcripts, and GitHub code repositories, into a common vector index, and answers questions against one, several, or all of a user’s sources through a single pipeline that performs semantic-lexical reranking, enforces per-source citation diversity on combined queries, gates its own confidence, and streams incremental answers with citations. It does not propose a new retrieval or generation model; three of its engineering decisions appear uncommon in this particular combination:

First, deterministic source-overview chunks: SAGE synthesizes one additional chunk per source, placed first, built purely from the source’s own statistics and excerpts with no LLM call, so broad questions such as “what is this about” are retrievable despite overlapping poorly with any single content chunk. Second, a reranking-

plus-diversity-floor combination discovered as a bug fix: ranking purely by blended score across a combined-source query can silently starve one source of any citation, so the remedy reserves each combined source’s single best chunk before filling remaining slots by score. Third, a confidence heuristic that gates refusal, not just a display badge: SAGE computes a scalar confidence score from retrieval evidence alone and uses it, before generation, to decide whether Grounded Mode should answer or refuse with a clarifying question, with a narrower exemption for video sources.

Section II situates SAGE against prior work. Section III gives its architecture, Section IV the four ingestion pipelines, Section V the source-overview mechanism, and Section VI the reranking and diversity-floor formulas. Section VII covers confidence, answer modes, and interaction; Section VIII reports validation, evaluation, and limitations; Section IX concludes.

II. Related Work

Dense retrieval and RAG. Dense retrieval runs from DSSM [27] to dual-encoder retrievers such as DPR [2], the design SAGE’s pipeline descends from. RAG [1] introduced conditioning a generator on retrieved passages; Fusion-in-Decoder [3] extended it by aggregating evidence over many passages jointly, close to SAGE’s multi-chunk context window. The broader space is catalogued by Gao et al. [4]. SAGE encodes with a distilled SBERT-family bi-encoder [5], a MiniLM checkpoint [6], [7] distributed as all-MiniLM-L6-v2 [8], trading accuracy for the latency an interactive product demands; on our evaluation set two newer small encoders brought no retrieval gain (Table IV).

Reranking, hybrid retrieval, and chunking. Cross-encoders [10] and late-interaction models such as ColBERT [11] improve on bi-encoder retrieval by scoring pairs jointly, at added cost. SAGE instead blends its bi-encoder score with a lexical term-recall score, following CLEAR [12], BM25 [9], and Reciprocal Rank Fusion [13], avoiding a second neural pass. Fixed-size chunking with overlap remains dominant, though studied directly for chunk-size sensitivity [14], context-preserving “late chunking” [15], and strategy comparisons [16]; SAGE applies a sentence-aware chunker to text and a separate band chunker to video (Section IV).

Modality-specific retrieval and confidence. Video QA [17], [18], retrieval-augmented code completion [19], [20], [21], and multi-document reading [22] each address one modality; SAGE trains none, instead normalizing all four into one embedding space (Sections V–VI). Gating refusal on retrieval evidence rather than generator certainty follows selective prediction under distribution shift [23], limits on self-reported confidence [24], and persistent hallucination [25]; retrieval augmentation separately lowers hallucination [26]. Consumer tools such as NotebookLM [28] and frameworks such as LangChain [29] and LlamaIn-

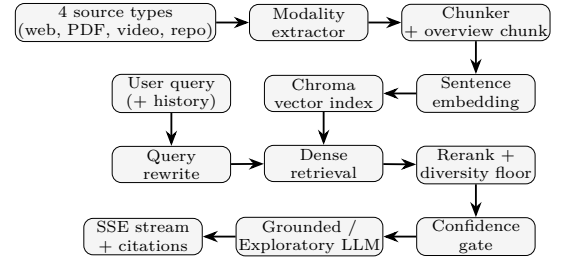


Fig. 1. SAGE end-to-end architecture: a shared ingest-chunk-embed-index contract feeds a single retrieve-rerank-gate-generate query pipeline.

dex [30] address an adjacent problem; SAGE is neither, so direct comparison lies outside this paper’s focus.

III. System Architecture

SAGE is a FastAPI backend with a React web client and a Chrome side-panel extension, backed by ChromaDB (vector store) and Supabase (relational store of record for sources, knowledgebases, and job status). Figure 1 shows the two halves: an ingestion half, where each source type is extracted by its own extractor into a common chunk representation, and a query half, where every question passes through one retrieval, reranking, confidence, and generation pipeline.

A source is one ingested website, PDF, video, or repository; sources are grouped into knowledgebases, and a query can target a single source, an entire knowledgebase, or an explicit list of combined source_ids spanning one or more knowledgebases. Every chunk, regardless of modality, carries a common metadata schema: a knowledgebase and source identifier, a source type, a chunk index, and a content-addressed chunk identifier computed as the truncated SHA-256 hash of the knowledgebase id, source id, chunk index, and chunk text, which makes re-ingestion of unchanged content naturally idempotent at the vector-store upsert layer.

A. Persistence and Metadata

Two stores back every source, kept in intentional sync rather than merged into one. ChromaDB holds only what retrieval needs: chunk text, embedding vector, and a flat metadata dictionary (Chroma rejects None-valued fields, so the upsert layer strips any unset field first). Supabase holds the relational structure the product needs but retrieval does not: knowledgebase and source records, titles, URLs, timestamps, and, for asynchronous source types, job status rows polled through /jobs/{job_id}. A source’s chunk_kind field lets one shared context-assembly routine (Section VI) special-case ordinary content chunks versus the source-overview chunk described next, without knowing which modality produced either. The background job tracker is a plain in-process dictionary with no persistence or expiry, a prototype simplification revisited in Section VIII.

Table I lists the system’s REST surface.

TABLE I
SAGE API surface (/api/v1 prefix)

Method	Path	Purpose
GET	/health	liveness check
POST	/qa/ask, /qa/ask/stream	sync QA / SSE streaming QA
POST	/ingest/website,	sync crawl / PDF
	/ingest/pdf	upload
POST	/ingest/video	async transcript-API /
	(-auto-transcribe)	Whisper fallback
POST	/jobs/github-ingest	async, only GitHub ingest path
GET	/jobs/{job_id}	poll async job status
GET	/sources(/grouped),	list sources / overview
	/sources/{id}/summary	text
DELETE	/sources/{id}	delete source, its vectors
POST	/sources/website/recrawl	async re-crawl of a source

Two details shape the rest of this paper. GitHub ingestion has no synchronous endpoint; it is always dispatched as a background job polled via `/jobs/{job_id}`, unlike synchronous website and PDF ingestion. Video ingestion is exposed as two independent, explicitly user-triggered endpoints rather than one endpoint with an automatic fallback: `/ingest/video` attempts the transcript API and fails distinguishably if none exists, and the client separately offers a “transcribe automatically” action calling `/ingest/video-auto-transcribe`, with no server-side chaining between the two.

IV. Multi-Source Ingestion

Each modality has its own extractor, but every extractor ends at the same handoff point: a list of `TextChunk` objects with metadata, ready for embedding and upsert into the vector store. Table II summarizes the four pipelines; the remainder of this section describes each in turn.

Website. The production website path is a headless Playwright crawl performing a breadth-first, same-domain traversal, re-validating every followed link (including redirect targets) against a public-address check to close a redirect-to-internal-address SSRF bypass. Extracted text passes through the shared sentence-aware chunker below; websites are deduplicated on a normalized URL (stripped `www.`, ports, and tracking parameters).

PDF. PDF upload is validated by a %PDF magic-byte check and capped at 25 MB. Extraction uses `PyMuPDF`’s per-page text; an empty extraction (a scanned, image-only PDF) fails explicitly with an “OCR is not implemented” error rather than silently indexing nothing. Deduplication is content-addressed on the SHA-256 hash of the uploaded bytes, invariant to renaming.

Video. Given a YouTube URL, the primary path calls the YouTube transcript API for timestamped caption segments, which a dedicated chunker greedily accumulates toward a 900-character target within a 700–1000-character band. When no transcript exists, the user can explicitly trigger a local Whisper (faster-whisper) fallback over downloaded audio, capped at 900s duration and 100 MB. Both paths preserve per-chunk timestamps, later

surfaced as `[MM:SS–MM:SS]` citation labels; videos are deduplicated on their canonical YouTube identifier.

GitHub repository. A repository is fetched via a shallow git clone `–depth 1`, falling back to a safely-extracted archive ZIP with zip-slip protection. The cloned tree is scanned file by file, skipping binaries and common non-source directories, and bounded by five caps (300 files, 1200 chunks, 2,000,000 characters, 250 KB/file, 100,000 lines/file) so a large monorepo degrades to a partial index rather than failing. Chunking targets 40–120 line ranges rather than a character budget, keeping chunks near function/block scale; repositories are deduplicated on canonical owner/repo.

Shared chunker and idempotency. Website, PDF, and (with a distinct parameterization) video text pass through a common sentence-aware chunker: sentences accumulate into a chunk until a 1200-character budget would be exceeded, then a new chunk opens seeded with the prior chunk’s last 180 characters, so a fact split across a boundary is not orphaned. This chunk-size/overlap band is studied for sensitivity elsewhere [14]; SAGE fixes it globally rather than adapting it per document type (Section VIII). Every chunk’s identifier is the truncated SHA-256 hash of its knowledgebase id, source id, index, and text, so re-ingesting unchanged content produces byte-identical ids and idempotent upserts with no separate change-detection check. A website’s recrawl path does not rely on this: it deletes and unconditionally re-ingests the entire knowledgebase, favoring simplicity over the cost of re-embedding unchanged pages.

V. Source-Overview Chunks

A short, non-specific question such as “what is this website about” or “what does this repo do” has poor term overlap, and often poor semantic similarity, with any single content chunk, because the answer lives in the source’s structure as a whole. Rather than add a second LLM call to summarize each source at ingestion time (added cost, latency, and hallucination risk before a user ever asks a question), SAGE constructs one additional, deterministic overview chunk per source from the source’s own statistics and excerpts, always inserted as chunk index 0.

Every overview chunk shares a small set of primitives: a term-frequency pass over a hand-built stopword list keeps the six most frequent remaining terms as a topic line (deliberately lexical rather than TF-IDF or embedding-based, since the goal is retrievability by a vague query, not a precise summary), and a first/last excerpt helper gives the chunk a sense of beginning and end. Its underlying text carries an internal marker stripped before display, and is exempt from the per-chunk truncation limit applied during context assembly (Section VI).

Website and PDF. The website overview draws on the crawl’s title, start URL, pages attempted versus crawled, chunk count, up to five title - url rows, and pooled top

TABLE II
Per-modality ingestion summary

Modality	Extraction method	Chunking unit	Chunk size band	Notable caps
Website	Playwright multi-page crawl (BFS)	sentence-aware, char budget	1200 chars / 180 overlap	same-domain only; SSRF-checked per hop
PDF	PyMuPDF (fitz) per-page text	sentence-aware, char budget	1200 chars / 180 overlap	25 MB upload; born-digital text only
Video	YouTube transcript API / Whisper (faster-whisper)	segment accumulation	900 target / 700–1000 chars	900 s duration, 100 MB audio (auto path)
GitHub repo	git clone -depth 1 (ZIP fallback)	line-range accumulation	40–120 lines	300 files, 1200 chunks, 2,000,000 chars, 250 KB/file

terms. The PDF overview draws on filename, page count, character count, chunk count, and a lightweight heading detector over the first three pages (capped at six headings).

Video. The overview draws on title, video identifier, duration, and transcript origin (native captions versus Whisper), reusing the already-chunked transcript chunks to render [MM:SS–MM:SS] text rows for the first and last three chunks, with a synthetic “Full video” timestamp label distinguishing it from ordinary segment chunks.

GitHub repository. This overview is qualitatively different: it is filesystem-introspective, re-walking the cloned directory on disk rather than deriving everything from chunk metadata. It lists top-level directories, checks for roughly ten on-disk conventions (README, dependency manifest, tests, entry points, routes, models), and renders a depth-two ASCII directory tree alongside file-extension and indexed/skipped counts - a structurally rich, directly verifiable codebase summary produced without any model inference.

VI. Retrieval, Reranking, and the Diversity Floor

Retrieval. A query (after the rewriting step described in Section VII) is embedded with the same all-MiniLM-L6-v2 sentence encoder used at ingestion, with L2-normalized output vectors, and searched against Chroma’s HNSW index configured for cosine distance. The candidate pool size is not fixed: for a GitHub-only query, the pool is $\min(2k, 8)$ where k is the requested top- k (default 6, capped at 6 for GitHub regardless of the client’s request); for other single-source queries it is $2k$ uncapped; and when a query explicitly combines $n \geq 2$ source_ids, the pool is widened further to $2k \cdot n$, capped at 40, specifically so that the diversity floor below has enough real per-source candidates to reserve from, rather than reserving from an already-truncated top- k list dominated by one source.

Reranking. Each retrieved chunk’s raw semantic score is $s = \text{clip}_{[0,1]}(1 - d)$, where d is Chroma’s returned cosine distance. This is combined with a lexical term-recall score $\ell = |Q \cap T|/|Q|$, where Q is the set of lowercased query tokens of length ≥ 3 (alphanumeric plus underscore, minus a small stopword list) and T is the corresponding token set of the chunk text; that is, the fraction of query terms that literally appear in the chunk, not a symmetric Jaccard overlap. The final rerank score is the fixed linear blend

$$\text{score} = 0.78s + 0.22\ell, \quad (1)$$

Algorithm 1 Diversity-floor selection

Require: ranked chunks C sorted by score desc.; requested k ; combined source ids S , $|S| \geq 2$
 $\triangleright$ reserved slots, one per source

- 1: $R \leftarrow \emptyset$
- 2: for $c \in C$ do
- 3: if $c.\text{source} \notin \{r.\text{source} : r \in R\}$ and $|R| < k$ then
- 4: $R \leftarrow R \cup \{c\}$
- 5: end if
- 6: end for
- 7: $F \leftarrow$ next highest-scored chunks in $C \setminus R$, filling until $|R \cup F| = k$
- 8: return $\text{sort_by_score_desc}(R \cup F)$

rounded to four decimal places. This gives semantic similarity the dominant weight while still rewarding exact keyword matches - useful for proper nouns, error codes, or API names that a bi-encoder alone may not distinguish sharply. Unlike a cross-encoder reranker [10] or ColBERT-style late interaction [11], Equation 1 requires no additional forward pass through a neural network at query time; both terms are available from the retrieval step and a cheap token-set intersection.

The diversity floor. Reranking chunks purely by Equation 1 and truncating to top- k has a specific failure mode when a query explicitly combines multiple sources: if source A has several moderately good chunks and source B has one excellent chunk, a pure score-sort can push B ’s only relevant chunk outside the cutoff, so the answer silently omits a source the user explicitly asked about. Found during manual testing of combined-source queries, this is now handled by a diversity floor (Algorithm 1), which activates only when a request explicitly names two or more combined source_ids; a single source or whole-knowledgebase reference uses plain score ranking.

The reserved set guarantees each combined source at least one citation slot before any slots are filled by pure score; remaining slots are filled by score across all sources, and the combined selection is re-sorted and truncated to k . The floor guarantees representation rather than proportional representation, since reserving more than one slot per source would starve capacity for whichever source is actually most on-topic.

Context assembly. Selected chunks are assembled under two character budgets: a total-context budget (12,000 characters, 10,000 for all-GitHub) and a per-chunk budget (1,600, or 1,200 for GitHub). Overview chunks are capped

at twice the per-chunk budget, so an unusually long overview cannot crowd real evidence out of the context. After scoring, each selected source’s overview is always appended (for up to three sources), and when a question asks about a source’s beginning or end (“what is his advice at the end?”), its first or last content chunks are appended too, since semantic search cannot see position. Appending after scoring keeps κ (Equation 2) a measure of retrieval strength alone. Chunks are appended in retrieval order and tagged with a citation marker ([S1], ...); a chunk exceeding the remaining budget is truncated but still cited, while chunks beyond a fully exhausted budget are dropped from both context and citations. GitHub chunks carry a direct blob/...#L{start}-{end} permalink, and video chunks an [MM:SS–MM:SS] label, both on the structured Citation object.

VII. Confidence, Answer Modes, and Interaction

SAGE offers two answer modes. Grounded Mode is constrained to answer only from the retrieved context and to reply “Not available in the provided source” when the context does not contain the answer; Exploratory Mode always attempts an answer (at a higher generation temperature, 0.3 versus 0.1) and is intended for brainstorming over the ingested sources rather than fact lookup. Retrieval evidence is summarized in a confidence score

$$\kappa = \text{clip}_{[0,1]}(0.65b + 0.35\bar{a} + 0.025\min(n, 5)), \quad (2)$$

rounded to two decimals, where b is the top chunk’s blended score (Equation 1), $\bar{a}$ the mean of the top three, and n the number retrieved; the last term is a small coverage bonus (up to 0.125 for five or more chunks), reflecting that several chunks in rough agreement are weaker-but-real evidence.

An earlier design refused before any generation call whenever $\kappa < 0.28$ (video excepted). In testing, low κ more often signalled a wording mismatch than missing evidence: paraphrased, misspelled, and non-English questions scored low yet were answerable from the retrieved chunks. SAGE therefore refuses outright only when nothing is retrieved; otherwise the generator judges the evidence under the grounded prompt, and answers with $\kappa < 0.45$ carry an explicit verify-citations warning. Section VIII measures the resulting refusal behaviour.

A separate heuristic computes the user-facing confidence label (“High”/“Medium”/“Low”), not simply a bucketing of κ : vague or short queries and fewer than three retrieved chunks cap the label at “Medium,” a strong-match shortcut forces “High,” and otherwise κ is thresholded at 0.75/0.45, values calibrated in Section VIII. The label is deliberately decoupled from the refusal gate, since a single number could not both gate whether to answer and explain to a user why an answer should be trusted without one purpose degrading the other.

Query rewriting. Every question is rewritten before retrieval by a single temperature-0, 80-token-capped call that fixes spelling, expands abbreviations, translates non-English questions into English (the language the corpus is indexed in, Section IV), and resolves references using the last three conversation turns. The turns are passed as quoted context rather than as chat messages, since as chat turns the model tended to answer the question instead of rewriting it; the selected sources’ titles are passed too, so “who came up with it?” or “tldr” resolve to the source being asked about. Retrieval then searches with both the original and the rewritten query and keeps each chunk’s best semantic score, so exact identifiers in the user’s own wording are never lost to a paraphrase; a timeout falls back silently to the original question.

Generation providers. All LLM calls (rewriting, translation, answering) run on free-tier APIs through an ordered fallback chain: Google Gemini 2.5 Flash for answers and 2.5 Flash-Lite for rewriting and translation, then Groq-hosted open models (Qwen3, gpt-oss-20b, gpt-oss-120b). Each model has an independent quota; a rate-limit response benches only that model, for the reset time the provider reports (an hour for an exhausted daily quota), and the next model answers immediately. If every model is briefly exhausted, a request waits at most 20 s for capacity before returning a retry notice instead of failing.

Streaming and memory. The streaming endpoint (/qa/ask/stream) shares its entire pipeline with the synchronous endpoint, so the two cannot silently diverge in what evidence they retrieve. It emits a meta event (confidence, citations, follow-up), a sequence of delta token events, and a final done event; a mid-stream timeout still emits done with the partial answer plus an error notice. Conversation history feeds both query rewriting and a keyword-matched follow-up generator on refusal; optional Supabase history persistence runs on a bounded 2-second timeout, so a slow write can only cost a warning, never a failed answer. Every request also logs five timing components via a monotonic counter, observational rather than an aggregated benchmark.

VIII. Validation, Evaluation, and Limitations

Test suite as evidence. SAGE’s backend test suite comprises 120 test functions across 16 files: chunk ordering, GitHub URL parsing and repository overviews (including a guard against symlinks that point outside a cloned repository), an end-to-end API integration suite, reranker and confidence checks, per-modality overview metadata, URL deduplication, vector-store edge cases, video timestamp and transcript handling, the LLM provider fallback chain, and the evaluation scorer. All 95 offline tests pass; the integration suite additionally needs live network services. This is validation evidence; retrieval and answer quality are measured separately below.

The diversity-floor discovery. The clearest evidence for the diversity floor’s necessity is its own discovery history:

it was added only after manual combined-source testing surfaced answers citing only one of two explicitly selected, genuinely relevant sources, exactly the failure mode of Section VI. A purely score-ranked reranker looks correct in isolation; the defect surfaces only once combined multi-source querying is tested directly.

Limitations. PDF extraction has no OCR path and fails cleanly on scanned documents. The Whisper transcription fallback requires an explicit user action rather than happening automatically. The confidence formula (Equation 2) was designed by hand; only its label thresholds are calibrated, on a small self-authored set (Table VI). The background job tracker for asynchronous ingestion is in-process and non-persistent, so status is lost on a backend restart. Website recrawling deletes and fully re-embeds a knowledgebase rather than diffing old and new content, wasting computation on unchanged pages. Finally, the Chrome extension covers websites, YouTube videos, and GitHub repositories but not PDF upload, which exists in the React web client alone.

Evaluation set. To move beyond code-level validation, we built a 50-question evaluation set over six real, previously ingested sources: this paper’s PDF, a TED talk transcript (Tim Urban, “Inside the Mind of a Master Procrastinator”), the kennethreitz/records GitHub repository, the Wikipedia article on RAG, a college website crawl, and python.org/about. Questions deliberately span factual (17), typo-ridden (5), Hindi or Hinglish (5), paraphrased (4), vague (4), summary (2), multi-source (2), and positional (1) phrasings, plus 10 questions whose answers are not in the selected source. Ground truth is a short evidence string per question (e.g., “one box for every week”), matched case- and whitespace-insensitively against chunk text, so labels survive re-embedding and re-ingestion; every evidence string was verified to exist in its source, and strings occurring in many chunks were replaced with more specific ones. Vague and summary questions have no single correct chunk and are scored on answers only, leaving 34 retrieval-scored questions. The set and harness are in the repository (backend/eval).

Retrieval ablation. Table III switches pipeline stages on one at a time through the production code path. Context recall is the share of questions whose evidence reaches the generator’s context; Recall@ k counts only the top- k reranked chunks; MRR is the mean reciprocal rank of the first evidence-bearing chunk. The BM25 blend of Equation 1 leaves recall unchanged but lifts MRR from 0.69 to 0.77, i.e., it moves the right chunk higher. Always-included overviews and positional chunks add the remaining recall (91% to 94%). Query rewriting shows no retrieval gain on this set with MiniLM, and a small MRR cost; its value appears in answers (Table V), where typo, Hindi, vague, and follow-up phrasings are all answered correctly, and in retrieval with another encoder (bge-small: 88% to 91%).

Embedding model. Re-embedding all 340 stored chunks

TABLE III
Retrieval ablation, 34 questions, all-MiniLM-L6-v2

Configuration	Ctx. recall	Recall@ k	MRR
Dense only	91%	91%	0.69
+ BM25 hybrid rerank	91%	91%	0.77
+ Query rewriting (dual query)	91%	91%	0.75
Full (+ overview, position)	94%	91%	0.75

TABLE IV
Embedding models, full pipeline, 34 questions

Model	Ctx. recall	Recall@ k	MRR
all-MiniLM-L6-v2	94%	91%	0.75
bge-small-en-v1.5	91%	91%	0.70
multilingual-e5-small	88%	88%	0.71

into separate collections (no re-crawling) allowed a like-for-like encoder comparison on the full pipeline (Table IV). Neither newer small encoder improved retrieval on this data, so SAGE keeps all-MiniLM-L6-v2. With 34 questions one question is about 3 points, so we read this as “no measurable gain,” not as MiniLM being generally superior.

Answers and refusals. All 50 questions were then answered end to end by the full system in Grounded Mode, on the free-tier provider chain (Section VII), at a median of 2.4s per answer (Table V). An answer counts as correct if it contains an expected key term; for unanswerable questions, only a refusal counts. SAGE refused all ten unanswerable questions and hallucinated none. Of the four answerable misses, one is a retrieval failure (the confidence-formula passage of this paper is never retrieved), two are generator errors with the evidence present in context (“How long was the senior thesis?” was refused although “90-page senior thesis” was retrieved; a question about the database library was answered without naming SQLAlchemy), and one reflects an ill-posed question of ours (it asks about calendar “rows,” which the talk never describes).

Confidence calibration. Table VI relates κ to correctness on the answerable questions. Answers with $\kappa \geq 0.75$ were correct 94% of the time versus 84% for $0.60 \leq \kappa < 0.75$, so the “High” label now starts at 0.75 (previously 0.60) and “Medium” at 0.45, aligned with the verify-citations warning. The relationship is only moderate: every answerable question scored $\kappa \geq 0.4$, the lowest bucket holds five questions, and the misses occurred at κ between 0.64 and 0.75, because they were generator errors rather than weak retrieval. κ is therefore best read as retrieval strength, a necessary but not sufficient condition for a correct answer.

These results come from a self-authored set of 50 questions over six sources, with keyword-based answer scoring, run once on free-tier models whose provider can change between calls; they measure SAGE on realistic, messy phrasings of real sources, not against public benchmarks or competing systems, which remains future work (Section IX).

TABLE V
Full-system answers, 50 questions (Grounded Mode)

Metric	Value
Answer accuracy (40 answerable)	90%
type / Hindi / paraphrase / vague	100% each
summary / multi-source / positional	100% each
factual	76%
Refusal recall (10 unanswerable refused)	100%
Refusal precision (3 false refusals)	77%
Median answer latency	2.4s

TABLE VI
Confidence calibration, 40 answerable questions

κ range	n	Accuracy
0.40–0.60	5	100%
0.60–0.75	19	84%
≥ 0.75	16	94%

IX. Conclusion

SAGE shows that a single ingest-chunk-embed-index-retrieve-rerank-generate contract can span four structurally unrelated source types without collapsing into a lowest-common-denominator design, provided the points where modalities genuinely differ, overview-chunk construction, context budgets, and per-source citation fairness, are explicit, modality-aware decisions rather than implicit assumptions. The diversity-floor discovery is offered as a concrete failure mode for other multi-source RAG builders to test for directly: a purely score-ranked reranker looks correct until exercised against genuinely combined queries. Future work includes OCR-enabled PDF ingestion, persistent background job scheduling, adaptive per-modality chunk sizing, and evaluation against public RAG benchmarks with larger, independently labeled question sets.

References

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems* (NeurIPS), 2020. arXiv:2005.11401.
- [2] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781. arXiv:2004.04906.
- [3] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proc. EACL*, 2021. arXiv:2007.01282.
- [4] Y. Gao et al., “Retrieval-augmented generation for large language models: A survey,” 2023. arXiv:2312.10997.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019. arXiv:1908.10084.
- [6] W. Wang et al., “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *NeurIPS*, 2020. arXiv:2002.10957.
- [7] W. Wang et al., “MiniLMv2: Multi-head self-attention relation distillation for compressing pretrained transformers,” 2020. arXiv:2012.15828.
- [8] Sentence-Transformers, “all-MiniLM-L6-v2 model card,” Hugging Face. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [9] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [10] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” 2019. arXiv:1901.04085.
- [11] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proc. 43rd Int. ACM SIGIR Conf.*, 2020. arXiv:2004.12832.
- [12] L. Gao, Z. Dai, and J. Callan, “Complement lexical retrieval model with semantic residual embeddings,” 2020. arXiv:2004.13969.
- [13] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759.
- [14] A. Jimeno-Yepes et al., “Financial report chunking for effective retrieval augmented generation,” 2024. arXiv:2402.05131.
- [15] M. Günther et al., “Late chunking: Contextual chunk embeddings using long-context embedding models,” 2024. arXiv:2409.04701.
- [16] C. Merola and J. Singh, “Reconstructing context: Evaluating advanced chunking strategies for retrieval-augmented generation,” in *2nd Workshop on Knowledge-Enhanced Information Retrieval*, ECIR, 2025. arXiv:2504.19754.
- [17] J. Lei et al., “TVQA: Localized, compositional video question answering,” in *Proc. EMNLP*, 2018. arXiv:1809.01696.
- [18] C. Zhang et al., “A simple LLM framework for long-range video question-answering,” in *Proc. EMNLP*, 2024. arXiv:2312.17235.
- [19] H. Husain et al., “CodeSearchNet challenge: Evaluating the state of semantic code search,” 2019. arXiv:1909.09436.
- [20] F. Zhang et al., “RepoCoder: Repository-level code completion through iterative retrieval and generation,” in *Proc. EMNLP*, 2023. arXiv:2303.12570.
- [21] D. Wu et al., “Repoformer: Selective retrieval for repository-level code completion,” in *Proc. ICML*, 2024. arXiv:2403.10059.
- [22] D. S. Sachan et al., “End-to-end training of multi-document reader and retriever for open-domain question answering,” in *NeurIPS*, 2021. arXiv:2106.05346.
- [23] A. Kamath, R. Jia, and P. Liang, “Selective question answering under domain shift,” in *Proc. ACL*, 2020. arXiv:2006.09462.
- [24] S. Kadavath et al., “Language models (mostly) know what they know,” 2022. arXiv:2207.05221.
- [25] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, art. 248, 2023.
- [26] K. Shuster et al., “Retrieval augmentation reduces hallucination in conversation,” in *Findings of EMNLP*, 2021, pp. 3784–3803. arXiv:2104.07567.
- [27] P.-S. Huang et al., “Learning deep structured semantic models for web search using clickthrough data,” in *Proc. 22nd ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2013, pp. 2333–2338.
- [28] Google, “NotebookLM: How to try Google’s experimental AI-first notebook,” Google Blog, 2023. [Online]. Available: <https://blog.google/technology/ai/notebooklm-google-ai/>
- [29] LangChain, Inc., “LangChain documentation.” [Online]. Available: <https://docs.langchain.com/>
- [30] LlamaIndex, “LlamaIndex documentation.” [Online]. Available: <https://developers.llamaindex.ai/python/framework/>